Project Post-Mortem: GAN Smart Cube 3D Tracker & CFOP Trainer

1. Introduction

The goal of this project was to connect a physical GAN 356 i3 Smart Cube to a computer and render its movements in real-time in a 3D web environment. We eventually expanded this into a professional-grade Competition Timer and CFOP Algorithm Trainer with holographic move hints.

2. Phase 1: The Python Era & Reverse Engineering

Initially, we attempted to build a backend bridge using Python to read the Bluetooth data and send it to a web frontend.

The Tools Used

- **nRF Connect & BLE Scanner (Mobile):** Used to discover the GAN cube's Bluetooth Low Energy (BLE) services and characteristics. We identified the primary data service UUID (`6e400001-...`).
- **Wireshark:** Used for Bluetooth packet sniffing to see what data the cube was broadcasting when a face was turned.
- **Python (`bleak`):** An asynchronous Bluetooth library used to connect to the cube from a desktop.

The Roadblocks & Decoding Techniques

When we successfully connected to the cube via Python, we discovered the data was heavily encrypted.

1. **The MAC Address Key:** We learned that GAN cubes use their own MAC address as a seed to generate the decryption key. We had to reverse the bytes of the MAC address and inject them into a base key.
2. **The AES-CBC Wall:** The payload was encrypted using AES-128-CBC. We attempted to use Python's `cryptography` library to decrypt the hex streams. However, GAN frequently updates their firmware, changing the initialization vectors (IVs) and S-Boxes. Our Python scripts kept failing with padding errors or spitting out garbage data because the underlying hardware logic was too obfuscated.

3. Phase 2: The Breakthrough

Instead of reinventing the wheel and manually fighting GAN's AES encryption, we searched for existing open-source implementations.

We found a compiled, minified React application (`iamthecu.be / gan-i3-356-bluetooth` library) that already contained the "Holy Grail": a custom-built, JavaScript-native AES decryption engine with the exact S-Boxes and IV arrays needed for modern GAN cubes.

The Pivot to Web Bluetooth

Finding this library allowed us to completely abandon the clunky Python backend. Modern browsers (Chrome/Edge) support the **Web Bluetooth API**. We realized we could run the entire application—Bluetooth connection, decryption, and 3D rendering—100% client-side in a single HTML file.

4. Phase 3: Building the 3D Engine (Three.js)

With the Bluetooth data finally decrypted into clean strings (e.g., `"U" , "R'"`), we built the visualizer.

Constructing the Cube

Instead of loading a 3D model, we built the cube procedurally using **Three.js**.

- **Structure:** We created a 3x3x3 grid using nested `for` loops (x, y, z from -1 to 1). We skipped the very center `(0,0,0)` . This gave us 26 distinct `THREE.Group` objects (8 corners, 12 edges, 6 centers).
- **The Black Plastic & Stickers:** To mimic the beautiful aesthetic of real cubes, we used HTML5 `<canvas>` . We drew a black square, overlaid a smaller, rounded, colored square, and exported that as a `THREE.CanvasTexture` . This gave the 3D meshes realistic plastic gaps and perfectly rounded stickers.

The Rotation Mathematics

Rotating a 3D Rubik's cube in code is notoriously difficult due to floating-point drift and grouping issues.

1. **Dynamic Grouping:** When a move like `"R"` is received, the code doesn't rely on hardcoded arrays. Instead, it gets the *current world position* of all 26 pieces (`getWorldPosition`). It grabs the 9 pieces where `x === 1` and attaches them to a temporary `THREE.Group` .
2. **Easing:** We used a mathematical easing function `1 - Math.pow(1 - progress, 4)` (Ease-Out Quart) to make the turns feel snappy and physical, rather than robotic and linear.
3. **Matrix Realignment:** After the animation completes, floating-point math can leave a piece at `x = 0.9999998` . If left uncorrected, the cube will eventually explode. We force every

piece to `Math.round()` its position and rotation to the nearest 90 degrees (`Math.PI / 2`), update its local matrix, and then destroy the temporary group.

Camera Orbit (Quaternions)

To allow the user to drag and rotate the whole cube, we captured mouse delta movements. Instead of using standard Euler angles (which cause "Gimbal Lock"—where axes overlap and the camera flips wildly), we applied the rotations using `THREE.Quaternion` .

5. Phase 4: State Machines & Logic Algorithms

We implemented a robust State Machine to handle Competition and Trainer modes.

The Variables

- `timerState` : Transitions between `idle` → `scrambling` → `inspection` → `running` → `finished` .
- `targetSequence` : An array of strings (e.g., `["R", "U", "R'"]`).

The Mistake Tracker & Double Move Forgiveness

If a user is supposed to execute `R2` but only turns `R` , a naive program would mark it as a mistake. Our logic intercepts this:

```
if (target.endsWith('2') && move[0] === target[0] && !move.endsWith('2')) {
    // They did half the move. Update the target to require one more turn.
    targetSequence[sequenceIndex] = move;
}
```

If a genuine mistake is made, the move is pushed into a `wrongMoves` array. The user is forced to perform the exact reverse move (calculated by stripping or adding a `'` prime symbol) before they can continue.

6. Phase 5: The "Neon Cage" Hologram Effects

To make the app look professional, we designed a holographic guidance system.

Additive Blending

Normal 3D objects block light. To create a "Hologram", we used `THREE.MeshBasicMaterial` with `blending: THREE.AdditiveBlending` and `depthWrite: false` . This tells the GPU to *add* the colors of the hologram to the pixels behind it, creating an emissive, glowing effect that looks like pure light.

The 3D Arrow Math

To draw a curved arrow that hugs the face of the cube:

1. **The Arc:** We used `THREE.TorusGeometry`, which naturally creates a donut. We only rendered a segment of the donut (e.g., `Math.PI * 0.8` for a 90-degree turn).
2. **The Arrowhead:** We created a `THREE.ConeGeometry`. To place it at the exact end of the Torus, we used basic trigonometry:
   - `x = radius * Math.cos(arcLength)`
   - `y = radius * Math.sin(arcLength)`
3. **Dynamic Flipping:** The Torus draws counter-clockwise by default. To make a clockwise arrow, we simply flipped the entire group on the X-axis (`scale.x = -1`).
4. **Pulsing Animation:** Inside the `requestAnimationFrame` loop, we used a Sine wave (`Math.sin(performance.now() * 0.005)`) to continuously modulate the opacity of the arrow and the intensity of a hidden `THREE.PointLight`, creating a realistic "breathing neon" effect.

7. Conclusion

By leveraging a pre-compiled AES decryption library and combining it with modern Web Bluetooth, HTML5 UI design, and advanced Three.js matrix mathematics, we successfully transformed a physical bluetooth toy into a professional, browser-native e-sports training platform.